

School of Computer Science

University of Windsor

Hashing

by

Adlane Habed

Review

Review

- Arrays, lists, queues, stacks and trees are used to store and retrieve records.
- A record looks like something like this

Student #: 145785
Name: Adelson-Velskii
CGPA: 9.5
Other information:

- Each record has a key value:

...Review

Binary search: key = 13

1	3	5	7	9	11	13	15	17	19	21
---	---	---	---	---	----	----	----	----	----	----

 3 comparisons

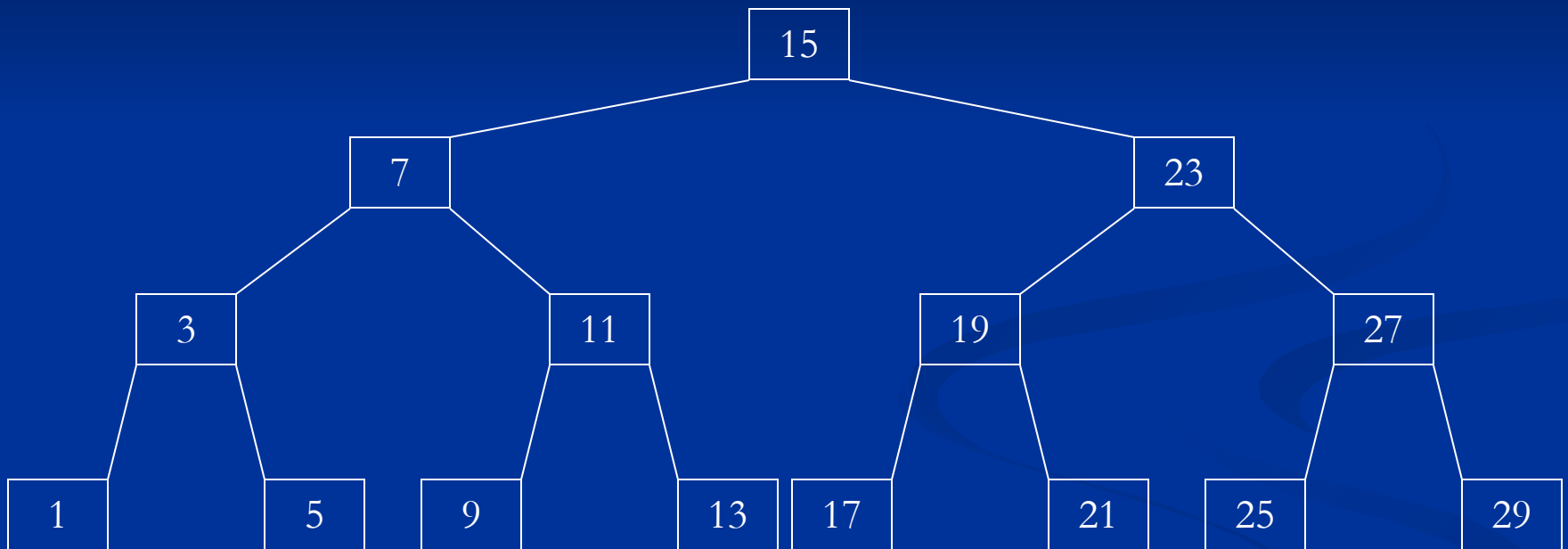
Sequential search: key = 13

1	3	5	7	9	11	13	15	17	19	21
---	---	---	---	---	----	----	----	----	----	----

 7 comparisons

...Review

Retrieve key=13 in a balanced Binary Search Tree



4 comparisons

...Review

Data structure	Complexity	
	$O(\log n)$	$O(n)$
Sorted array	search	insert, delete
Sorted linked-list		search, insert, delete
Balanced BST	search, insert, delete	

Agenda

- What is hashing?
- Hash functions
- Collision-resolution strategies
- Analysis
- Problems to think about

What is hashing?

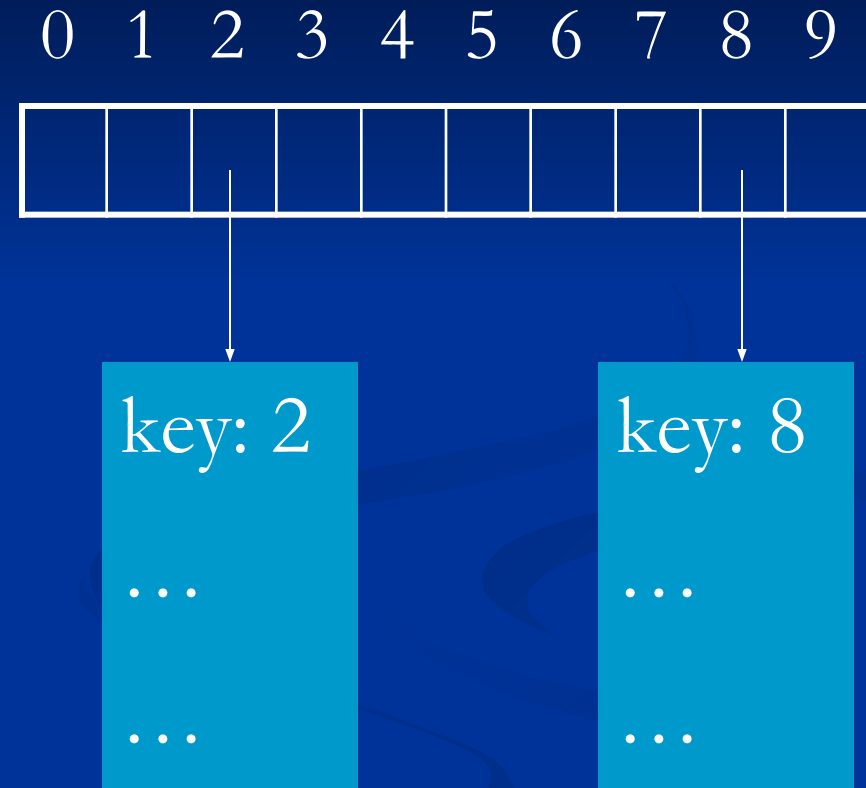
1. Basic idea
2. Definitions
3. Perfect hashing
4. Collisions
5. Open-addressing vs. Chaining

Basic idea

- A data structure that allows insertion, deletion and search in $O(1)$ in average.
- A data structure that requires a limited or no search in order to find a record.
- The location of the record is calculated from the value of its key.
- No order in the stored records.
- No findMin or findMax.

...Basic idea

- Consider records with integer key values:
0, 1, 2, 3, 4, 5, 6, 7, 8, 9
- Create a table of 10 cells:
index of each cell in the range [0..9].
- Each record is stored in the cell whose index corresponds to its key value.



Definitions

- **Hashing**

The process of accessing a record, stored in a table, by mapping the value of its key to a position in the table.

- **Hash function**

A function that maps key values to table positions.

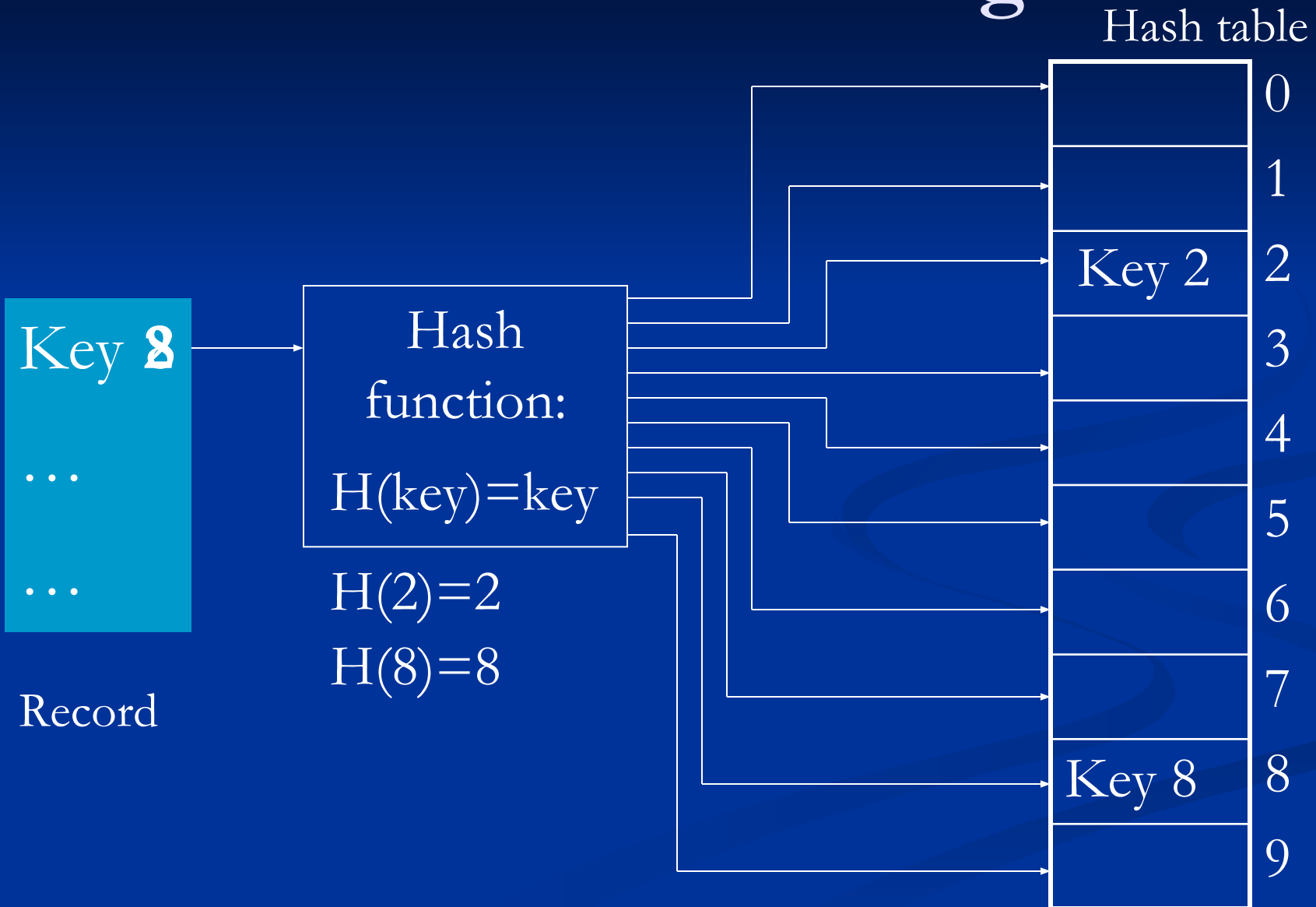
- **Hash table**

The array where the records are stored.

- **Hash value**

The value returned by the hash function. It usually corresponds to a position in the hash table.

Perfect hashing



...Perfect hashing

- Each key value maps to a different position in the table.
- All the keys need to be known before the table is created.
- **Problem:** what if the keys are neither contiguous nor in the range of the indices of the table?
- **Solution:** find a hash function that allows perfect hashing! Is this always possible?

...Perfect hashing

- **Example:** a company has 100 employees. Social Insurance Number (SIN) is used as a key for a each record.
- Given a 9 digits SIN, should we create a table of 1,000,000,000 cells for only 100 employees?
- Considering that the SI Numbers of all 100 employees are known in advance does not guarantee to find a perfect hash function.

...Perfect hashing

- **The birthday paradox:**

what is the number of persons that need to be together in a room in order to, “most likely”, have two of them with the same date of birth (month/day)?

Answer: only 23 people.

Hint: calculate p the probability that no two persons have the same date of birth.

...Perfect hashing

- Hash functions that allow perfect hashing are so rare that it is worth looking for them only in special circumstances.
- In addition, it is often that the collection of records is not known in advance.

Collisions

- What if we cannot find a perfect hash function?

Collision: more than one key will map to the same location in the table!

- Can we avoid collisions? No, except in the case of perfect hashing (rare).
- **Solution:** select a “good” hash function and use a collision-resolution strategy.

...Collisions

Example: The keys are integers and the hash function is
 $hashValue = key \bmod tableSize$

- If $tableSize = 10$, all records whose keys have the same rightmost digit have the same hash value.

Insert 13 and 23

0	1	2	3	4	5	6	7	8	9
			13						
23									

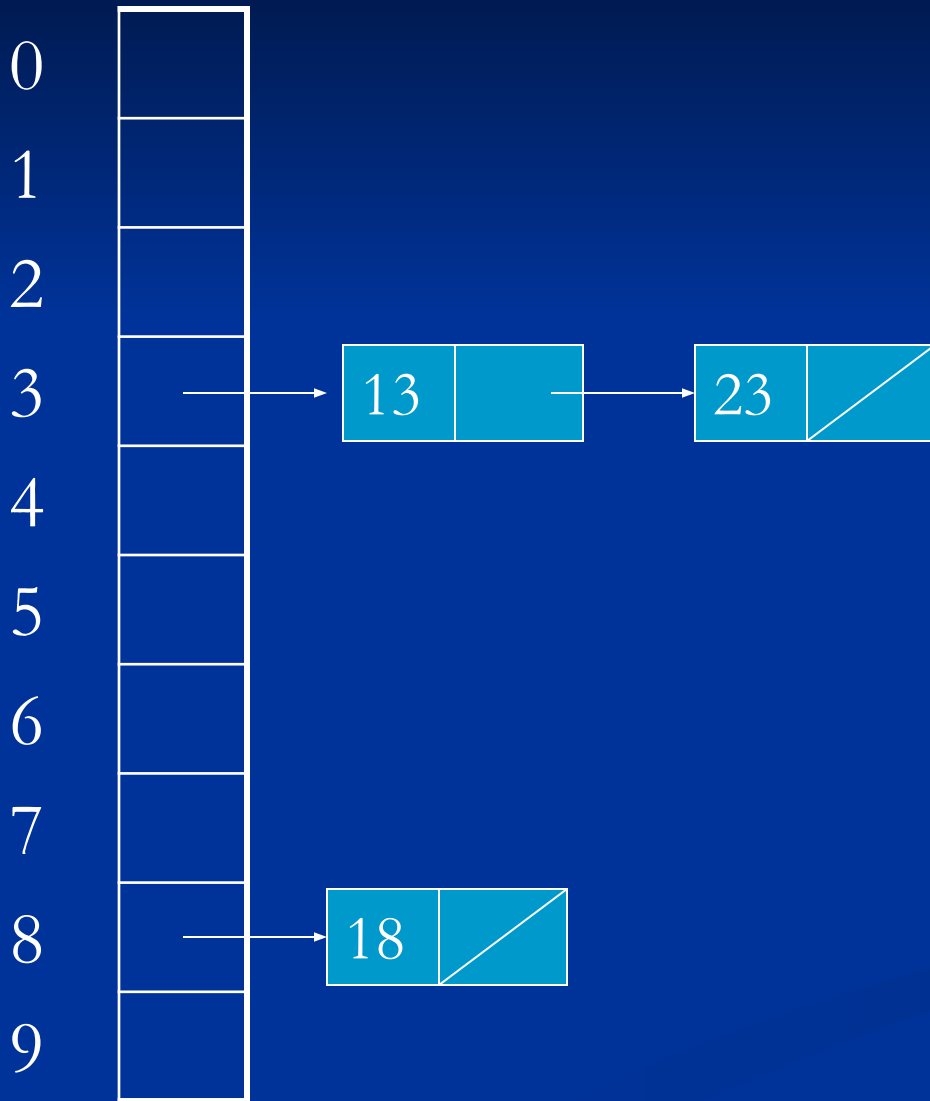
Open-addressing vs. chaining

- **Open-addressing:** Storing the record directly in the table.

Deal with collisions using collision-resolution strategies.

- **Chaining:** Each cell of the hash table points towards a linked-list.

...Chaining



$$H(key) = key \bmod tableSize$$

Insert 13

Insert 23

Insert 18

Collision is resolved by inserting the elements in a linked-list.

Hash functions

1. Hash functions
2. Division
3. Digits selection
4. Mid-square
5. Folding
6. String keys

Hash functions

- Can we have a hash function that avoids collisions?

Collisions are nearly unavoidable! If we are careful when selecting the hash function, then the number of collisions will be few.

- **Exception:** the hash function is selected for a specific set of records **Perfect hashing**

...Hash functions

- **A poor hash function:**

Maps keys non-uniformly into table locations, or maps a set of contiguous keys into clusters.

- **An ideal hash function:**

- Maps keys uniformly and randomly onto the entire range of table locations.
- Each location is equally likely to be used for a randomly chosen key.
- Fast computation.

Hash functions: division

- Division:

$$H(key) = key \bmod tableSize$$

$$0 \leq key \bmod tableSize \leq tableSize-1$$

Empirical studies have shown that this function gives very good results.

...division

- Assume $H(key) = key \bmod tableSize$
- All keys such that $key \bmod tableSize = 0$ map into position 0 in the table.
- All keys such that $key \bmod tableSize = 1$ map into position 1 in the table.
- This phenomenon is unavoidable for positions 0 and 1: **we wish to avoid this phenomenon when possible.**

...division

- Assume $tableSize = 25$
- All keys that are multiples of 5 will map into positions 0, 5, 10, 15 and 20 in the table!
- Why? because key and $tableSize$ have 5 as a common factor:

There exists an integer m such that:

$$key = m \times 5$$

Therefore, $key \bmod 25 = 5 \times (m \bmod 5)$ is a multiple of 5

... division

- Choose *tableSize* as a **prime number**.
- Example: *tableSize* = 29 (a prime number)

$$5 \bmod 29 = 5,$$

$$10 \bmod 29 = 10,$$

$$15 \bmod 29 = 15,$$

$$20 \bmod 29 = 20,$$

$$25 \bmod 29 = 25,$$

$$30 \bmod 29 = 1,$$

$$35 \bmod 29 = 6,$$

$$40 \bmod 29 = 11...$$

Hash functions: digit selection

- Digit(s) selection:

$$key = d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8 d_9$$

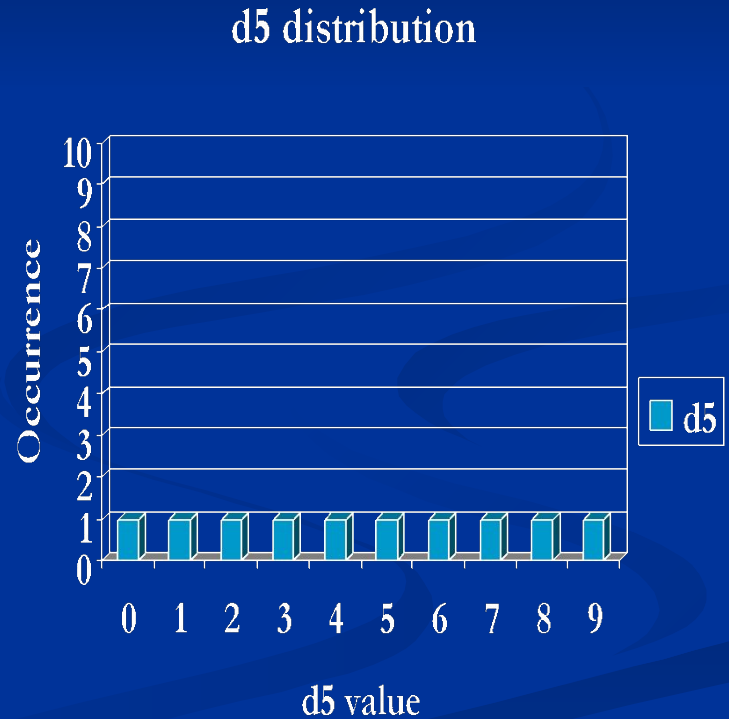
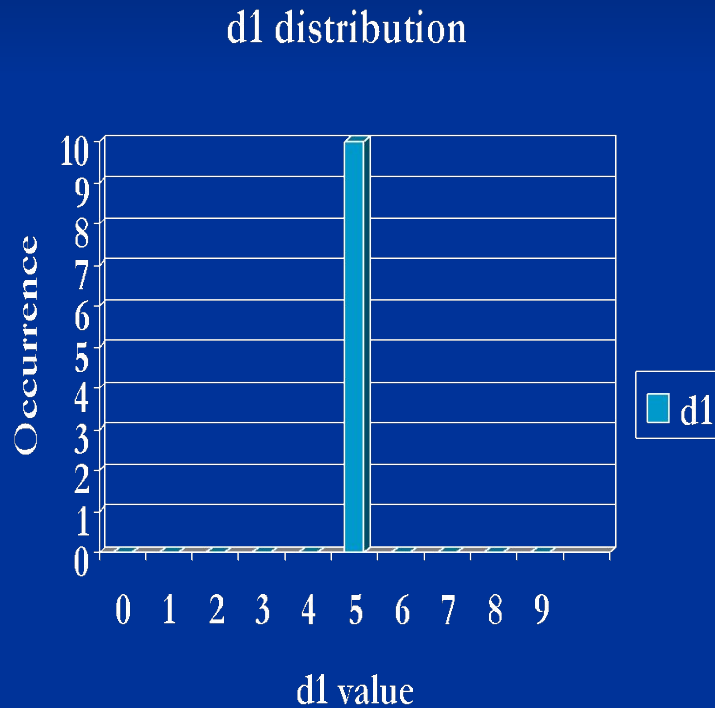
If the collection of records is known,

how to choose the digit(s)?

Analysis of the occurrence of each digit.

Digit selection: analysis

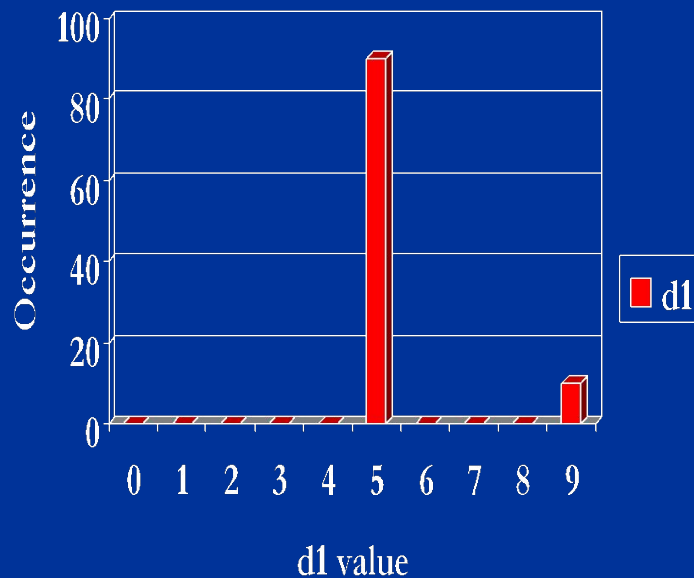
Assume 10 records are to be stored:



...Digit selection: analysis

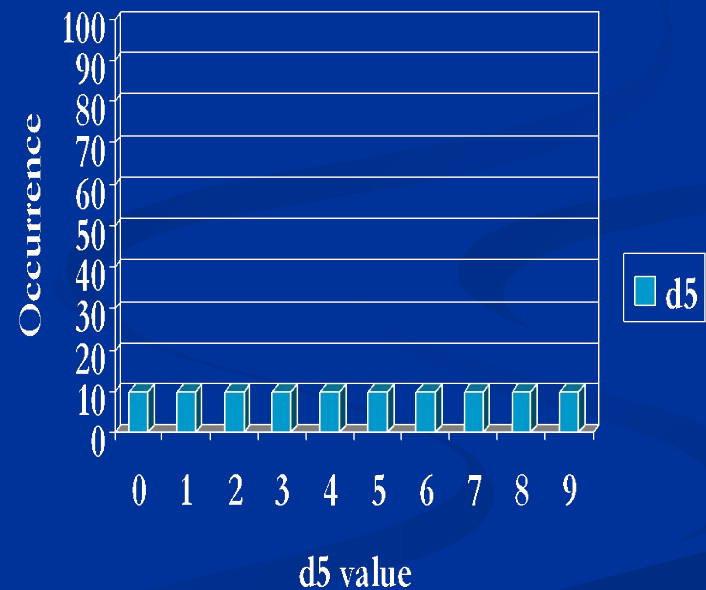
Assume 100 records are to be stored:

d1 distribution



Non-uniform distribution

d5 distribution



Uniform distribution

...Digit selection: analysis

- Consider the hash function:

$$H(d_1 d_2 d_3 d_4 d_5 d_6 d_7 d_8 d_9) = d_5 d_7$$

d_5 and d_7 are uniformly distributed

...but $d_5 = 3$ and $d_7 = 8$ appear very often in common!

38 is the only position used in the range 30...39 increasing the chances for collisions.

Analysis of correlation is required.

Hash functions: mid-square

- Mid-square: consider $key = d_1 d_2 d_3 d_4 d_5$

$$\begin{array}{r} d_1 d_2 d_3 d_4 d_5 \\ \times \quad d_1 d_2 d_3 d_4 d_5 \\ \hline \end{array}$$

$r_1 r_2 r_3 r_4 r_5 r_6 r_7 r_8 r_9 r_{10}$

Select middle digits, for example $r_4 r_5 r_6$

Why the middle digits and not leftmost or rightmost digits?

Mid-square: example

54321

×
54321

54321

108642

162963

217284

271605

2950771041

Only 321 contribute in the 3 rightmost digits (041) of the multiplication result.

Similar remark regarding the leftmost digits.

All key digits contribute in the middle digits of the multiplication result.

Hash functions: folding

- **Folding:** consider $key = d_1 d_2 d_3 d_4 d_5$

Combine portions of the key to form a smaller result.

In general, folding is used in conjunction with other functions.

Example: $H(key) = d_1 + d_2 + d_3 + d_4 + d_5 \leq 45$

or, $H(key) = d_1 + d_2 d_3 + d_4 d_5 \leq 207$

Folding: example

- Consider a computer with 16-bit registers, i.e. integers $< 2^{16} = 65536$
- Assume the 9-digit SIN is used as a key.
- SIN requires folding before it is used:

$$d_1 + d_2d_3d_4d_5 + d_6d_7d_8d_9 \leq 20007$$

The key is a string

- When the key is a string, the ASCII code of each character in the string is considered.
- The ASCII code is an integer value in the range 0...127.

String to decimal conversion:

Consider *key* = “data”

hashValue =

$$('a' + 't' \times 128 + 'a' \times 128^2 + 'd' \times 128^3) \bmod tableSize$$

...The key is a string

This method generates huge numbers that the machine might not store correctly.

- Goal: reduce the number of arithmetic operations and generate relatively small numbers.

hashValue = 'd' **mod** *tableSize*

hashValue = (*hashValue* × 128 + 'a') **mod** *tableSize*

hashValue = (*hashValue* × 128 + 't') **mod** *tableSize*

hashValue = (*hashValue* × 128 + 'a') **mod** *tableSize*

Collision-resolution strategies in open addressing

1. Linear probing:
The problem of clustering
2. Quadratic probing

Linear probing

If $H(key)$ is already occupied:

Search sequentially (and by wrapping around the table if necessary) until an empty position is found.

Example: $H(key) = key \bmod tableSize$

Insert 89 Insert 9 Insert 58 Insert 49 Insert 18

0 1 2 3 4 5 6 7 8 9

								18	89
--	--	--	--	--	--	--	--	----	----

58 9
49

...Linear probing

$$\textit{hashValue} = H(\textit{key})$$

Probe table positions

$$(\textit{hashValue} + i) \bmod \textit{tableSize}$$

with $i = 1, 2, \dots, \textit{tableSize} - 1$

Until an empty position is found in the table, or all positions have been checked.

Primary clustering

- Linear probing makes that many items are stored in a few areas creating clusters:

This is known as **primary clustering**.

- Contiguous keys are mapped into contiguous table locations.
- **Consequence:** Slow search even when the table's load factor λ is small:

$$\lambda = (\text{number of occupied locations}) / \text{tableSize}$$

Quadratic probing

- Collision-resolution strategy that eliminates primary clustering.
- It works as follows:

$hashValue = H(key)$

if $table[hashValue]$ is occupied

probe table positions

$(hashValue + i^2) \bmod tableSize, i=1,2,3...$

until an empty position is found.

...Quadratic probing

Quadratic probing creates spaces between the inserted elements hashing to the same position: eliminates primary clustering.

Insert 99

0	1	2	3	4	5	6	7	8	9
								18	89
								58	99

...Quadratic probing

- **Very important result:**

If quadratic probing is used, *tableSize* is prime and table is at least half empty, the insertion of a new element is guaranteed and no cell is probed twice.

Analysis

Analysis

We calculate the average number of comparisons to search successfully S and unsuccessfully U for a record given the load factor of the table.

Analysis

- U =unsuccessful search S =successful search
- H , is uniform
- Linear probing:
$$U = (1 + 1/(1-\lambda)^2)/2 \quad S = (1 + 1/(1-\lambda))/2$$
- Quadratic probing:
$$U = 1/(1-\lambda) \quad S = -(1/\lambda)\ln(1-\lambda)$$
- Chaining:
$$U = \lambda \quad S = 1 + \lambda/2$$

Comparison

	U		S
Linear probing	$\lambda = 0.1$	1.11	1.05
	$\lambda = 0.5$	2.50	1.5
	$\lambda = 0.9$	50.5	5.5
Quadratic probing	$\lambda = 0.1$	1.11	1.05
	$\lambda = 0.5$	2.00	1.38
	$\lambda = 0.9$	10.00	2.55
Chaining	$\lambda = 0.1$	0.1	1.05
	$\lambda = 0.5$	0.5	1.25
	$\lambda = 0.9$	0.9	1.45

Problems to think about

Proofs

- Proof of the birthday paradox.
- In quadratic probing:

$$pos_i = (H(key) + i^2) \bmod tableSize$$

Show that: $pos_i = (pos_{i-1} + 2i - 1) \bmod tableSize$

What is the advantage of this result?

Implementation issues

- Implementation of hash tables.
- Deletion in the case of open-addressing.
- How to keep a table at least half empty?
- Empirical evaluation of different hash functions for a particular problem.
- Empirical evaluation of probing strategies for a particular problem.

Other questions

- What is the relationship between the number of probes for an insertion and an unsuccessful search?